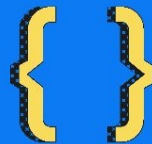


HaskellKatas: Hackathon para todos



Reynaldo Cordero |
Mercedes Cordero |
@HaskellKatas



HaskellKatas:

- **¿Qué es Haskell?**

- Haskell es un lenguaje de **programación funcional** y de **propósito general**, que fue desarrollado por primera vez en los años 80 por un grupo de investigadores liderados por el **matemático** estadounidense **Philip Wadler**. Haskell toma su nombre en honor al lógico Haskell Curry.
- Una de las principales características de Haskell es su enfoque en la **programación funcional**, lo que significa que se enfoca en la **evaluación de funciones** y en la manipulación de **datos inmutables**. Haskell es un lenguaje de programación **puramente funcional**, lo que significa que no tiene efectos secundarios, lo que lo hace muy adecuado para la programación de **sistemas complejos**.
- Haskell también se destaca por su **sistema de tipos avanzado**, que utiliza la **inferencia de tipos** para garantizar la **seguridad** del tipo de datos. Además, Haskell es un lenguaje de programación altamente **expresivo y conciso**, lo que significa que permite a los programadores escribir código que es **fácil de leer y mantener**.
- Haskell se utiliza en una amplia gama de aplicaciones, desde la **investigación** académica y la **educación** hasta la programación de **sistemas empresariales** y de **alta demanda**.



HaskellKatas:

- **¿Qué es una kata de programación?**

- Una kata de programación es un **ejercicio práctico** que se utiliza para mejorar las **habilidades de programación** de una persona. Es similar a una **rutina de entrenamiento** en artes marciales, en la que los practicantes practican una serie de movimientos una y otra vez para perfeccionar su técnica. En el caso de las katas de programación, los programadores practican la **resolución de problemas** y la **escritura de código** para mejorar su habilidad en un lenguaje de programación específico.
- Las katas de programación a menudo implican la resolución de un problema específico utilizando un **conjunto limitado** de herramientas o técnicas. Los programadores pueden trabajar en una kata de programación **solos** o en **grupos**, y pueden **compartir sus soluciones** y discutir diferentes enfoques para resolver el problema.
- Las katas de programación son una forma efectiva de mejorar las habilidades de programación de una persona, ya que permiten a los programadores practicar la resolución de problemas y la escritura de código de manera **sistemática y repetitiva**. Además, las katas de programación también pueden ser **divertidas y desafiantes**, lo que las convierte en una forma atractiva de mejorar las habilidades de programación.



HaskellKatas:

- ¿Qué es una HaskellKata?

- HaskellKatas es un método **sistemático** y **asistido** para **entrenar** la programación gracias a las características de **Haskell** (**expresivo, conciso, legible, matemático**).
- <https://gitlab.com/HaskellKatas/katas--proof-of-concept> **newk** (bash script)
- Se fundamenta en:
 - La puesta en **marcha** y los **pasos** a seguir son simples y rápidos.
 - Se trabaja con **gran cantidad** de código escrito por **otros**.
 - La esencia es **observar, pensar y relacionar** sobre la marcha.
 - La actividad principal es **kataficar** el código (**reordenar, reformatear, simplificar, refactorizar**) para que resulte **expresivo** o **artístico**.
 - No se requiere consultar información fuera, sólo **manejarla dentro**.
- **Calidad profesional: TDD** (desarrollo guiado por **pruebas**), control de versiones (git), **hackeable** ().



HaskellKatas:

- **¿Qué es un hackathon de programación?**
 - Se trata de un **evento** donde interesados en la programación pueden **compartir ideas** y **código** relacionados con retos de programación con el fin de hacer **avanzar un proyecto** y de **aprender** mutuamente.
 - Dado que las HaskellKatas requieren la **compartición** de abundante código de programación que sea **variado y diverso**, resultan ideales para ponerlas en práctica.
 - La otra forma de obtener decenas de soluciones de código listas para ser aprovechadas en una HaskellKata es hacer uso de **sitios web de desafíos de programación**.
 - <https://www.codewars.com/kata/50654ddff44f8002000000004/train/haskell>



HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### KATA

Multiply

<https://www.codewars.com/kata/50654ddff44f800200000004/train/haskell>

---- ##### NEWK

newk multiply <https://www.codewars.com/kata/50654ddff44f800200000004/train/haskell> 'Multiply'

---- ##### SRC

multiply :: Integer -> Integer -> Integer

multiply a b = **undefined**

---- ##### TEST

describe "multiply" \$ do

it "multiply 7 7 == 49" \$

multiply 7 7 `shouldBe` 49

it "multiply 11 11 == 121" \$

multiply 11 11 `shouldBe` 121



HaskellKatas: ¿Nos ponemos con las katas? ;-)

```
---- ##### QUICKCHECK TESTS
    it "...arbitrary numbers..." $ do
      property $ \ x y -> do
        multiply x y `shouldBe` (x * y)
```

```
---- ##### SOLUTIONS
multiply :: Int -> Int -> Int
multiply = (*)
```

```
----
multiply :: Int -> Int -> Int
multiply a b = a * b
```

```
----
multiply :: Int -> Int -> Int
multiply a b = head $ do
  return $ a * b
```



HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### SOLUTIONS

```
import Data.Maybe
```

```
multiply :: Int -> Int -> Int  
multiply a b = fromJust $ do  
  return $ a * b
```

```
multiply :: Int -> Int -> Int  
multiply a b  
  | b == 0 = 0  
  | b > 0 = a + multiply a (b - 1)  
  | b < 0 = multiply a (b + 1) - a
```



HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### SOLUTIONS

import System.IO.Unsafe

```
multiply :: Int -> Int -> Int
multiply a b = unsafePerformIO $ do
  return $ a * b
```

```
import Prelude hiding (return)
return = id
```

```
multiply :: Int -> Int -> Int
multiply a b = do
  return $ a * b
```



HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### SOLUTIONS

```
import System.IO.Unsafe (unsafePerformIO)
```

```
multiply :: Int -> Int -> Int
```

```
multiply a b =
```

```
  unsafePerformIO
```

```
    $ do
```

```
      putStrLn "Missile launched!"
```

```
      putStrLn
```

```
        $      "By the way: "
```

```
        ++ show a
```

```
        ++ " * "
```

```
        ++ show b
```

```
        ++ " = "
```

```
        ++ show result
```

```
      return result
```

```
where
```

```
  result = a * b
```



HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### SOLUTIONS

```
multiply :: Int -> Int -> Int
multiply a b = do
  (*) a b
```

```
multiply :: Int -> Int -> Int
multiply = \a    b  -> a * b
```

```
multiply :: Int -> Int -> Int
multiply a = (a*)
```

```
multiply = (*)
```



HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### SOLUTIONS

```
import Data.Functor.Identity (Identity)
```

```
multiply :: Int -> Int -> Identity Int
```

```
multiply a b = do
```

```
  return $ a * b
```

```
import Data.Functor.Identity
```

```
multiply :: Int -> Int -> Int
```

```
multiply a b = runIdentity $ do
```

```
  return $ a * b
```



HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### SOLUTIONS

import Data.Function

--import Data.Functor.Identity

multiply :: Int -> Int -> Int

multiply a b = a & (*) b

multiply :: Int -> Int -> Int

multiply 7 7 = 49

multiply 11 11 = 121

multiply 11 2 = 22

multiply a b = a * b

multiply :: Int -> Int -> Int

multiply a b = (*) a b



HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### SOLUTIONS

import Data.Maybe

```
multiply :: Int -> Int -> Int
multiply a b = fromMaybe 0 $ do
  return $ a * b
```

--import Data.Maybe

```
multiply :: Int -> Int -> Int
multiply a b = head (do return (a * b))
```

```
multiply :: Int -> Int -> Int
multiply 0 _ = 0
multiply _ 0 = 0
multiply a b = a * b
```

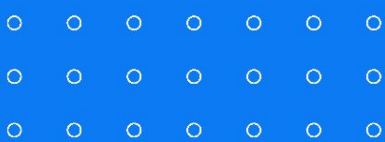


HaskellKatas: ¿Nos ponemos con las katas? ;-)

---- ##### SOLUTIONS

etc.



 {CODEMOTION} CONFERENCE
MADRID 2023

Gracias!

